ACTIVE: A Dynamic Pareto-Frontier-Based Unified Index for Dual-Vector Query

Shengyuan Lin
National School of Elite Engineers, Northeastern University, China
linsy5847@gmail.com

Shufeng Gong
Department of Computer Science and Engineering, Northeastern University, China
gongsf@mail.neu.edu.cn

Zheng Wang
Department of Computer Science and Engineering, Northeastern University, China
wangzheng123@mails.neu.edu.cn

Feng Yao
Department of Computer Science and Engineering, Northeastern University, China
yaofeng@stumail.neu.edu.cn

Yanfeng Zhang
Department of Computer Science and Engineering, Northeastern University, China
zhangyf@mail.neu.edu.cn

Ge Yu
Department of Computer Science and Engineering, Northeastern University, China
yuge@mail.neu.edu.cn

ABSTRACT

Dual-vector queries (DVQ) have been increasingly adopted in real-world retrieval applications. In DVQ, object similarity is computed as a weighted combination of similarities over two vectors, where the weights are determined by a query-specific preference parameter. Pareto-frontier-based indices provide an effective solution for DVQ, as they achieve high search accuracy with low query latency. However, existing methods incur high construction overhead and do not support efficient dynamic updates. We identify two key challenges in handling static search and dynamic updates for dual-vector data. First, active range computation has a time complexity of $O(N^2)$, making it the main overhead in index construction. Second, no effective method exists for locally repairing Pareto frontiers in DVQ indices, and directly applying single-vector local repair strategies is ineffective in update throughput and index quality.

To address these issues, we propose ACTIVE , a dynamic Pareto-frontier-based DVQ index that supports efficient construction and updates while maintaining high retrieval performance under varying query preferences. To accelerate index construction, we design an angle-constrained dynamic pruning algorithm that reduces distance computations during active range computation. To support efficient updates, we introduce a connectivity-loss-driven adaptive repair strategy that selectively repairs affected nodes based on their connectivity loss. Furthermore, we maintain a reverse graph topology asynchronously to facilitate the localization and repair of affected nodes. Extensive experiments show that ACTIVE reduces index construction time by xx–yy× compared with the state-of-the-art Pareto-frontier-based index DEG, achieves 1.26–30.28× speedup in update throughput over baseline methods, and maintains comparable search accuracy under reinsertion-based updates.

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/iDC-NEU/ACTIVE.

1 INTRODUCTION

Approximate nearest neighbor search (ANNS) over high-dimensional vectors has been widely adopted in applications such as recommendation systems [10, 31, 39, 40] and information retrieval [2, 30, 52, 62]. In these applications, the unstructured attributes of an entity, such as image, text, or audio, are embedded into vectors, and objects whose vectors are close (similar) to the query vector are retrieved. Traditional ANNS primarily focus on single-vector retrieval [7, 15, 18, 21, 22, 28, 47, 56], i.e., entities are searched based on only *one* unstructured (vectorized) attribute.

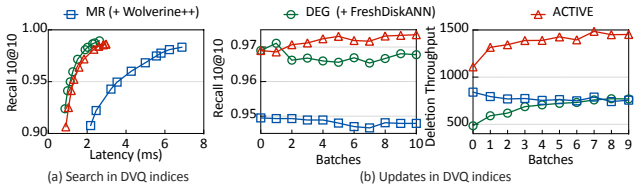


Figure 1: Performance of DVQ indices under static search and dynamic updates on the OpenImage dataset, where "MR" denotes multi-index retrieval.

However, many real-world retrieval tasks require jointly considering two vectorized attributes [3, 9, 29, 34, 36, 43, 45], with query-specific preferences assigned to each attribute. We refer to these tasks as the Dual-Vector Query (DVQ) problem. Given a query with two vectors, the distance between the query and an object is the weighted sum of their distances on the two vectors, where the query-specific preference determines the weight of each attribute distance. As query preferences vary, the distance between the same pair of entities changes accordingly, making efficient and accurate ANN retrieval particularly challenging. Furthermore, real-world datasets are inherently dynamic, with entities continuously being inserted and deleted. Such dynamic updates pose significant challenges to efficient index-based retrieval methods.

Example. In product search on e-commerce platforms, consider two users who upload images of a smartphone and provide textual descriptions of performance characteristics, such as "a smartphone with long battery life and excellent low-light photography". The first user aims to retrieve a smartphone that visually resemble the uploaded image, whereas the second user prioritizes the specific performance-related attribute. Accordingly, the first user assigns a larger weight to the image-vector distance, while the second user assigns a larger weight to the textual-vector distance when computing similarity based on the image and textual embeddings. Even when the two users provide the same phone image and textual

description, the different preferences can lead to different retrieval results. Meanwhile, vendors continuously update product listings due to inventory changes, ensuring accurate retrieval results for users. This makes the dynamic updating of dual-vector data indispensable. Thus, the DVQ problem requires efficient and accurate retrieval under arbitrary query preferences while also supporting efficient dynamic updates.

Motivation. Existing DVQ approaches [45, 46, 58, 60] mainly build graph-based ANNS indices over dual-vector data to enable efficient retrieval. However, they cannot simultaneously achieve high retrieval accuracy and high update efficiency. In general, these approaches can be divided into two categories.

1) Preference-specific multi-indices. These approaches [45, 46, 60] build single-vector indices by fixing the query preference in advance. A representative solution is multi-index retrieval (MR) [45, 60], which builds a separate index for each attribute vector. It searches the two indices independently, merges the retrieved candidates, and subsequently reranks them by computing the exact weighted distance. Since they assume that object distances are pre-determined, they suffer from limited retrieval quality under arbitrary query preferences, as shown in Figure 1a.

2) Preference-agnostic unified indices. Unlike existing single-vector indices [15, 21, 28] that select the similar vectors of each node as their neighbors, this approach [58] connects approximate nearest neighbors across all possible query preferences, ensuring that varying query preferences can be accommodated during search. To achieve this, approximate Pareto frontiers are utilized as the neighbors since they remain competitive under diverse preference settings, thereby enabling the selected neighbors to cover varying query preferences. As a result, Pareto-frontier-based indices provide a more suitable foundation for query-dependent similarity search in DVQ. However, existing unified indices do not support efficient dynamic updates because repairing Pareto frontiers is more complex than repairing neighbors in single-vector indices [26, 44, 59].

Our goal. In this work, we aim to design a dynamic Pareto-frontier-based DVQ index that supports efficient construction and updates while maintaining high retrieval performance under varying query preferences.

Challenges. Designing a Preference-agnostic unified index is challenging for two main reasons.

1) Expensive Edge Active-Range Computation in Construction. Pareto-frontier-based indices [58] select neighbors to cover different query preferences, but not every selected edge is useful under every preference. Thus, each edge needs to precompute its active ranges of the preference, allowing it to be traversed only when needed and avoiding redundant distance computations. Employing Relative Neighborhood Graph (RNG) pruning [58] can compute such active ranges, but it has a roughly $O(N^2)$ time complexity and accounts for 67.14% of the construction time on the OpenImage dataset [35]. Therefore, designing an efficient active-range computation method remains challenging.

2) Inefficient Approximate Pareto Frontier Repair in Update. In Pareto-frontier-based indices, repairing the Pareto-frontier neighbors of affected nodes (i.e., in-neighbors of deleted nodes) after deletions is critical to preserving graph connectivity. Reinsertion-based global repair strategies [1, 28, 57] rediscover the neighbors of affected nodes by traversing from entry nodes, causing many distance computations and high update overhead. However, directly applying single-vector local repair strategies [44, 59] to Pareto-frontier-based DVQ indices is ineffective, as validated in Figure 1b, since they cannot guarantee that Pareto frontiers can be identified within the local neighborhood of DVQ indices. Therefore, designing an efficient local repair strategy for approximate Pareto frontiers while maintaining high retrieval accuracy remains challenges.

ACTIVE. To this end, we propose ACTIVE , a dynAmiC Pareto-fronTier-based unIfied DVQ indEx that supports dynamic updates while maintaining high retrieval quality under arbitrary query preferences. ACTIVE is built upon two techniques.

To accelerate index construction, we develop an angle-constrained dynamic pruning algorithm, AC-Prune, in which each neighbor is compared only with those satisfying the angle constraint under the triangle inequality. By restricting unnecessary comparisons, AC-Prune reduces pruning overhead while preserving effective navigation quality of the DVQ index across diverse query preferences.

For efficient updates, we introduce CA-PFR, a connectivity-loss-driven adaptive repair strategy that locally repairs approximate Pareto frontiers. It employs three progressively stronger modes, namely historical-Pareto-frontier, affected-layer and multi-layer repair, according to the severity of connectivity degradation of affected nodes. This allows us to restore the connectivity of the graph without full re-insertion, significantly reducing hybrid vector distance computations and improving update throughput. Compared with local repair approaches designed for single-vector indices, such as FreshDiskANN [44], CA-PFR also preserves higher retrieval quality in DVQ.

In summary, the main contributions of this work are:

(1) An angle-constrained dynamic pruning algorithm, AC-Prune, which accelerates active-range computation of edges without degrading retrieval quality, thereby enhancing the efficiency of both index construction and dynamic updates for DVQ indices (§ 4.2).
(2) An efficient deletion repair algorithm for dynamic DVQ indices, CA-PFR, which locally repairs approximate Pareto frontiers according to the severity of connectivity degradation of affected nodes. We further introduce a system-level optimization that accelerates the localization and repair of affected nodes (§ 5.2).
(3) Extensive experiments on diverse datasets show that, compared with state-of-the-art baselines, ACTIVE accelerates index construction with comparable search accuracy, while achieving higher update throughput and improved retrieval accuracy during updates (§ 6).

2 BACKGROUND AND MOTIVATION

In this section, we first define the Dual-Vector Query (DVQ) problem and review Pareto-frontier-based DVQ index construction as well as update methods for single-vector graph-based indices. We then analyze their drawbacks and present the motivation of this work.

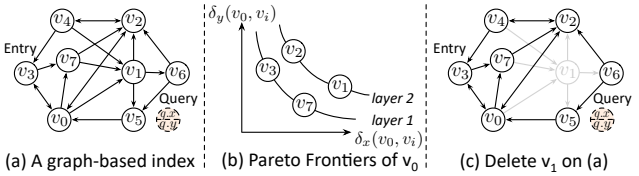


Figure 2: An example of a graph-based DVQ index. Each node maintains at most $R = 4$ neighbors. The dashed circle indicates a dual-vector query, v_3 represents the entry of the index, and v_5 represents the most similar node to the query.

2.1 Graph-based Dual-Vector Query

Let $O = \{o_0, o_1, \ldots, o_{N-1}\}$ denote a dataset of N objects, where each object $o_i \in O$ is represented by two feature vectors: (1) $o_i.x \in \mathbb{R}^{d_x}$, a d_x-dimensional vector capturing one attribute, and (2) $o_i.y \in \mathbb{R}^{d_y}$, a d_y-dimensional vector capturing another attribute.

Dual-Vector Query (DVQ). Given a query $q = \langle x, y, \alpha, k \rangle$, where $q.x \in \mathbb{R}^{d_x}$ and $q.y \in \mathbb{R}^{d_y}$ are the query vectors, $q.\alpha \in [0, 1]$ is a weighting parameter that quantifies the query preference, and $q.k$ is the number of objects to retrieve. Ideally, DVQ aims to retrieve the exact top-k nearest objects $R_{exact} \subset O$ for a query q, such that $\forall e \in R_{exact}, \forall o \in O \setminus R_{exact}$, we have $\text{Dist}(q, e) \leq \text{Dist}(q, o)$, where the distance [58] between q and an object o is defined as:

$$\text{Dist}(q, o) = q.\alpha \cdot \delta_x(q, o) + (1 - q.\alpha) \cdot \delta_y(q, o), \tag{1}$$

where $\delta_x(q, o) = \frac{\|q.x - o.x\|_2}{x_{\max}}$, $\delta_y(q, o) = \frac{\|q.y - o.y\|_2}{y_{\max}}$ are the normalized distance on each vector. Here, $\|\cdot\|_2$ denotes the Euclidean distance, and $x_{\max}$ (resp. $y_{\max}$) is the maximum Euclidean distance between any two x-vectors (resp. y-vectors) in the dataset, used for normalization. The parameter $q.\alpha \in [0, 1]$ controls the relative importance of the two attribute distances

However, the retrieval of R_{exact} requires exhaustive search over the entire dataset, which becomes computationally expensive and incurs high search latency, especially for dual-vector datasets [19]. To address this, DVQ in practice retrieves k approximate nearest results R_{DVQ}, achieving a tradeoff between retrieval accuracy and search efficiency [20, 58]. To quantify the quality of the approximate results R_{DVQ}, recall [24, 48] is commonly used, defined as $Recall@k = \frac{|R_{DVQ} \cap R_{exact}|}{k}$, where R_{exact} is the ground-truth set of the k closest objects to q. The goal of DVQ, as in single-vector ANNS, is to maximize recall while retrieving results efficiently.

Graph-Based Unified DVQ Index. Given a dual-vector dataset O and its corresponding graph-based index $G = (V, E)$, where each node $v \in V$ corresponds to an object $o_v \in O$, and the edges in E are constructed based on the specific distance rulers [15, 21, 28, 58] as shown in Figure 2a. In contrast to existing single-vector graph indices [15, 21, 28], the Graph-based unified DVQ index [58] constructs edges by selecting the Pareto frontiers of each node as its outgoing neighbors. The detailed construction procedure of Pareto-frontier-based indices is presented in Section 2.2.

2.2 Pareto-Frontier-Based Index Construction

Due to diverse query α in DVQ, Pareto-frontier-based indices select the Pareto frontiers of each node as its out-neighbors, which can nearly cover the approximate nearest neighbors under for all possible α values.

Algorithm 1: GPS($G, q, EP, ef_{construct}$)

Input: Graph $G = (V, E)$, query node q, entry node set EP, candidate pool size $ef_{construct}$
Output: Multi-layer Pareto frontiers $\{PF_q^1, \ldots, PF_q^l\}$ of q

```
Res ← FindPF(EP, ef_construct);
while |Res| < ef_construct do
    NNS ← unexplored nodes from the lowest layer in Res
          that contains such nodes;
    if NNS = ∅ then
        break;
    Add unvisited out-neighbors of nodes in NNS to Res;
    Res ← FindPF(Res, ef_construct);
return Res;
```

Pareto Frontier [27]. The construction of Pareto-frontier-based indices can be viewed as finding the Pareto frontier for each node. Specifically, given a node $v \in V$ for which we aim to find its neighbors, each candidate node $p \in V \setminus \{v\}$ induces a distance pair relative to v, $(\delta_x(v, p), \delta_y(v, p))$. For any $p, q \in V \setminus \{v\}$, we say that p *dominates* q, denoted by $p \prec q$, if and only if (1) $\forall i \in \{x, y\}$, $\delta_i(v, p) \leq \delta_i(v, q)$, and (2) $\exists j \in \{x, y\}$, $\delta_j(v, p) < \delta_j(v, q)$. The Pareto frontier of v is defined as $PF_v \subseteq V \setminus \{v\}$, where $\forall p \in PF_v, \nexists q \in V \setminus \{v\}, q \prec p$, i.e., the neighbors of v in PF_v are not able to be dominated by any node.

Multi-layer Pareto Frontiers. In Pareto-frontier-based indices, only using the Pareto frontier of each node as its out-neighbors leads to an extremely sparse graph index. This is because, in a dataset with millions of objects, the size of the Pareto frontier is typically small (e.g., the average size is 8.26 on the CC3M dataset) , while hundreds of candidate objects are usually required to select the final out-neighbors. To address this, the multi-layer Pareto frontiers $\{PF_v^1, \ldots, PF_v^l\}$ are selected as the candidate out-neighbors of each node v, where l is the number of layers bounded by the candidate pool size $ef_{construct}$, as shown in Figure 2b. Neighbors in the first layer correspond to the global Pareto frontier. Each subsequent layer is recursively constructed by removing the current Pareto frontier and extracting a new Pareto frontier from the remaining candidates, referred to as FindPF [5].

Approximate Pareto Frontier Search. However, the construction of exact multi-layer Pareto frontiers is computationally expensive, as it requires computing distances to all other nodes in the dataset for each node. Therefore, approximate Pareto frontiers are typically used as candidate neighbors for each node, which are constructed using the Greedy Pareto Frontier Search (GPS) to expand candidate neighbors layer by layer (see Algorithm 1). Specifically, GPS iteratively expands unexplored nodes in the lowest layer of Pareto frontiers containing them until no unexplored nodes remain. Finally, the candidate set Res is selected as the approximate Pareto frontier. Unlike exact Pareto frontier construction for a given node, GPS only evaluates nodes visited during the Greedy Pareto Frontier Search, thereby significantly reducing the computational cost.

Active Range Computation (Pruning). need to be polished GSFSince the out-neighbors of each node cover the nearest neighbors varying query preferences, a significant portion of the outgoing edges are unnecessary to be traversed for a given α value. To accelerate query processing, we assign preference intervals (i.e., *active*

Algorithm 2: Prune(v, C, R, τ)

Input: Node v to be pruned, candidate set C, maximum edges R, threshold τ to prune edges with a small use range

Output: Neighbor set NS

```
NS ← ∅;                                          // Retained nodes
foreach f ∈ C do
    r_f ← ∅;
    foreach e ∈ NS do
        compute the pruning range r_e;
        r_f ← r_f ∪ r_e;
    u ← [0,1] \ r_f;                             // Active ranges
    if len(u) ≥ τ then
        NS.add(f, u);        // len(u) = Σ_{[u_i,u_j]∈u} (u_j − u_i)
    if |NS| ≥ R then
        break;
return NS;
```

ranges) to each edge, enabling the system to selectively traverse only the edges whose active ranges include the query α. However, computing the active range of each edge is difficult [58].DEG adopts an RNG-inspired pruning strategy similar to those used in single-vector graph indices. Specifically, for each candidate edge, it examines its relationship with every other candidate edge and computes an interval r_f of the parameter α over which the edge becomes the longest edge in the corresponding triangle configuration. The active range is then obtained by excluding all such intervals from the complete parameter domain $[0, 1]$. This method requires comparing each neighbor of a node with all retained neighbors to determine the active ranges of edges, resulting in a quadratic time complexity of ($O(N^2)$) and introducing an additional substantial computational overhead.

Summary. Existing methods for DVQ index construction still face performance challenges, and thus require efficient pruning techniques to accelerate construction while preserving index quality.

2.3 Existing Single-Vector Graph-Based Indices Update

In real-world scenarios, timely updates of indices are essential for search accuracy. For example, after deleting node v_1, the target node v_5 becomes unreachable from the entry point v_3, as illustrated in Figure 2c. Although no specialized DVQ index has yet been proposed to efficiently support dynamic updates, existing update techniques for single-vector graph-based indices are already well-established [1, 44, 57, 59]. Next, we review representative update methods in existing single-vector graph indices.

Insertion Operation. When a new node v is inserted, existing methods first perform a greedy search to obtain candidate out-neighbors and then prune them to construct the final out-neighbor set $N_{out}(v)$. Finally, for each neighbor $v' \in N_{out}(v)$, a reverse edge (v', v) is added to ensure that newly inserted nodes remain reachable during subsequent insertions.

Deletion Operation. Existing repair strategies can be broadly classified into the following two categories: 1) *Reinsertion-Based Repair (RBR)* methods [1, 28, 57] treat affected nodes as newly inserted nodes and reinsert them to reconstruct their out-neighbors. Although these methods can maintain high index quality, the global search incurs significant computational overhead. 2) *Neighbor-Reconnection-Based Repair (NRBR)* methods [44, 59] restore graph connectivity through local reconnections, based on the intuition that neighbors of neighbors are likely to remain close to the affected node. For example, FreshDiskANN [44] establishes new edges between the in-neighbors and out-neighbors of the deleted node. Compared with RBR, NRBR may degrade index quality while achieving higher update throughput. However, in Pareto-frontier-based DVQ indices, these NRBR methods cannot guarantee that the neighbors of a node's neighbors belong to its Pareto frontiers, making it difficult to maintain high retrieval accuracy.

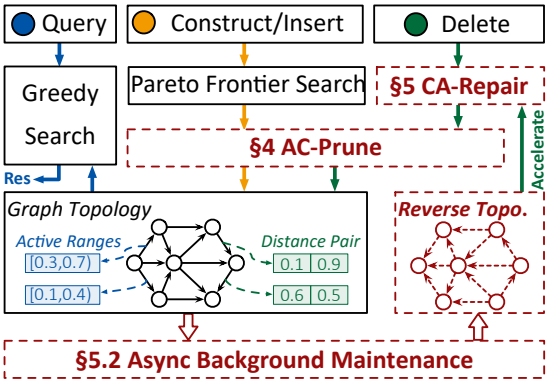


Figure 3: Workflow of ACTIVE .

Summary. Update strategies for single-vector graph indices provide useful insights for DVQ index updates. For instance, the insertion operation can largely reuse techniques from incremental construction. Nevertheless, efficient updates for DVQ indices remain challenging. In particular, designing an efficient deletion repair strategy that preserves high index quality remains challenges.

Limitions Based on the analysis in Section 2.2 and Section 2.3, existing Pareto-frontier-based DVQ indices suffer from inefficient index construction and substantial update overhead.

3 OVERVIEW

To overcome the above limitations, we design ACTIVE , a dynamic Pareto-frontier-based unified DVQ index that supports dynamic updates while maintaining high retrieval performance under arbitrary query preference α. To accelerate index construction, 1) we introduce an *angle-constrained active range computation* (§ 4) strategy that avoids comparing each candidate neighbor against all retained neighbors by leveraging angular constraints, thereby reducing unnecessary triangle inequality evaluations and distance computations. To accelerate index updates, 2) we propose a *connectivity-loss-driven adaptive repair* (§ 5) strategy that locally repairs the approximate Pareto frontiers of affected nodes according to the severity of their connectivity degradation. This approach substantially restores the connectivity of the graph index while maintaining high update throughput. When implementing the query system, We further incorporate *asynchronous reverse graph maintenance* to efficiently locate and identify nodes affected by deletions (§ 5.2).

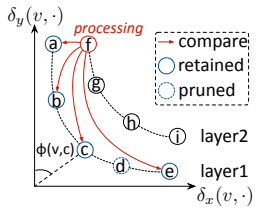


Figure 4: Illustration of the existing pruning algorithm [58].

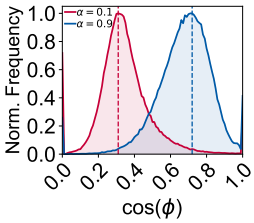


Figure 5: Distribution of retrieved nodes across different Pareto angles ϕ

Workflow. ACTIVE supports efficient index construction and update operations for Pareto-frontier-based DVQ indices, as well as efficient query processing.

Index Construction. ACTIVE employs an incremental construction strategy for each object in the dataset. Specifically, when constructing the out-neighbors of node v, the candidate set C is formed by collecting its approximate Pareto frontiers from the current index. Subsequently, C is pruned using our proposed AC-Prune to obtain the final neighbors, and the active range of each outgoing edge is computed. Finally, reverse edges pointing from the selected neighbors to node v are added to guarantee the reachability of v during subsequent searches.

Index Update. For insertion, ACTIVE adopts the incremental construction approach to add new nodes into the graph index. For deletion, ACTIVE first leverages reverse graph topology to quickly locate the affected nodes. Then, ACTIVE adaptively repairs the affected nodes with three methods, namely historical-Pareto-frontier repair, affected-layer repair, and multi-layer repair, according to the severity of connectivity degradation for each affected node. Finally, ACTIVE prunes the candidate neighbor set of each affected node to obtain its final neighbors.

Dual-Vector Query. For a given dual-vector query q and a query preference α, ACTIVE performs the greedy search based on the distance function (e.g., Eq. 1) to obtain the top-k approximate nearest neighbors. Specifically, ACTIVE maintains a candidate queue Q_c and a result queue Q_r, i.e., Q_c contains the visited node but not expanded while Q_r contains the expanded nodes, both ordered by their distances to the query, where Q^t and Q^b denote the first and last nodes in queue Q, respectively. At each iteration, Q_c^t is selected for expansion, i.e., its out-neighbors whose active ranges contain the query preference parameter α are inserted into Q_c. The expanded node is then inserted into Q_r. During the search, $|Q_r|$ is bounded by ef_{search}. The search terminates when $\text{Dist}(q, Q_c^t) > \text{Dist}(q, Q_r^b)$.

4 ANGLE-CONSTRAINED ACTIVE RANGE COMPUTATION

The workflow reveals that active range computation is invoked during both index construction and updates. This additional overhead substantially degrades both construction and update efficiency. To reduce this cost, we exploit the visited characteristics of Pareto-frontier-based neighbors and introduce an angle-constrained pruning algorithm that significantly improves computational efficiency.

4.1 Pareto Angle

During pruning, DEG computes the active ranges of all out-edges of a node in a nested-loop manner. Specifically, when pruning the candidate neighbors of node v, DEG sequentially traverses its out-neighbors $(a, b, c, d, e, f, \dots)$ to compute their active ranges, as illustrated in Figure 4. When traversing node f to compute its active ranges, DEG compares it against all previously retained nodes (i.e., final neighbors), namely a, b, c, and e. However, this strategy performs redundant comparisons for each candidate neighbor, e.g., node f. It is unnecessary to compare f with e, since f is close to the $\delta_y(v,\cdot)$ axis while e lies closer to the $\delta_x(v,\cdot)$ axis, making it highly unlikely that they will be simultaneously retrieved during query processing. To better describe our observation, we introduce the following definition.

Definition 1 (Pareto Angle (ϕ)). *Given a node v and its out-neighbor c, the Pareto angle $\phi(v,c)$ is defined in the 2D space induced by $(\delta_x(v,c), \delta_y(v,c))$. We define*

$$\phi(v,c) = \arccos\left(\frac{(\delta_x(v,c), \delta_y(v,c)) \cdot (0,1)}{\sqrt{\delta_x(v,c)^2 + \delta_y(v,c)^2}}\right), \tag{2}$$

where $\phi(v,c)$ denotes the angle between the vector $(\delta_x(v,c), \delta_y(v,c))$ and the positive $\delta_y(v,\cdot)$-axis. The value of $\phi(v,c)$ lies in $[0, \pi/2]$.

Observation. We construct a Pareto-frontier-based index on CM dataset, where all edges are assigned an active range of $[0,1]$. We perform search under query $\alpha = 0.1$ and $\alpha = 0.9$, and obtain the result queue Q_r of each query, which contains the ef_{search} closest retrieved nodes for the query. We further compute the Pareto angle $\phi(v,r)$ for each retrieved node $r \in Q_r$, where v is the in-neighbor that triggers the insertion of r into the candidate queue during the expansion of v. After the search, we compute the frequency distribution of all retrieved nodes across different cosine values $\cos(\phi)$. From Figure 5, we make two key observations:

1) Retrieval Probability Diversity. Under a given α, nodes with different Pareto angles exhibit different probabilities of being retrieved during the search. For example, when $\alpha = 0.1$, node f in Figure 4 is more likely to be retrieved than node e.

2) Pareto-Angle Locality. Retrieved nodes exhibit strong locality in the Pareto angle space, where high-frequency Pareto angle regions form contiguous intervals. For instance, when $\alpha = 0.1$, nodes with higher retrieval frequency are mainly distributed in the range of $\cos(\phi) \in [0.2, 0.4]$.

Intuition. Based on the two observations, we conclude that each neighbor does not need to be compared against all previously retained nodes when computing its active range. Instead, comparisons should be restricted to neighbors with small differences in Pareto angle values. This is because different neighbors of v (e.g., e and f) exhibit significantly different probabilities of being retrieved under a given preference parameter α, making it unlikely that the triangle (v, e, f) is formed during the search, thus invalidating the active range derived from such triangles. Therefore, we only consider those neighbors that are likely to jointly form triangles with it during the search, i.e., nodes close to it in the Pareto angle space.

Algorithm 3: AC-Prune $(v, C, R, \tau, \theta_{th})$

Input: Node v to be pruned, candidate set $C = \{PF_v^1, \ldots, PF_v^l\}$, maximum number of edges R, threshold τ to prune edges with a small use range, and angular threshold $\theta_{th} \in [0, \frac{\pi}{2}]$

Output: Neighbor set NS

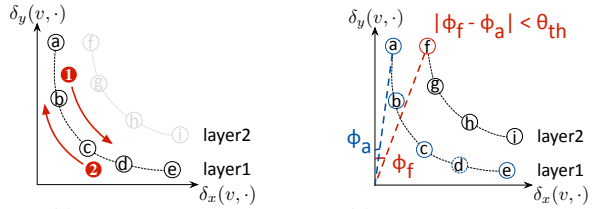
1 $NS \leftarrow \emptyset$; // Retained nodes
2 **foreach** $PF^i \in C$ **do**
3 **if** $i = 1$ **then** // The first layer
4 $LocalNS \leftarrow Prune(v, PF^1, R, \tau)$; // Forward order
5 $last\_node \leftarrow$ Pop_last($LocalNS$);
6 $NS \leftarrow \{last\_node\}$;
7 $NS \leftarrow NS \cup$ Prune$(v, PF^1 \setminus NS, R, \tau)$; // Reverse order
8 **else** // The remaining layers
9 **foreach** $f \in PF^i$ **do**
10 $r_f \leftarrow \emptyset$;
11 **foreach** $e \in NS$ **do**
12 **if** $|\phi(v, e) - \phi(v, f)| \leq \theta_{th}$ **then**
13 compute the pruning range r_e;
14 $r_f \leftarrow r_f \cup r_e$;
15 $u \leftarrow [0, 1] \setminus r_f$; // Active ranges
16 **if** $len(u) \geq \tau$ **then**
17 NS.add(f, u);
18 **if** $|NS| \geq R$ **then**
19 **break**;
20 **return** NS;

4.2 Active Range Computation Based on Pareto Angle Difference

Based on the above intuition, we propose an angle-constrained dynamic pruning algorithm, *AC-Prune*, to reduce distance computations during the pruning phase, thereby significantly improving pruning efficiency while maintaining search performance comparable to existing pruning algorithms. The core intuition of AC-Prune is that, for any two candidate neighbors e and f of a node v, if both are retrieved during the search under a given α, the absolute difference between their Pareto angles, i.e., $|\phi(v, e) - \phi(v, f)|$, should not be excessively large.

Specifically, when pruning the candidate neighbor set of a node v, AC-Prune adopts different processing strategies for candidate neighbors in different layers (i.e., the first layer and the remaining layers), as illustrated in Algorithm 3. This is because nodes in the first layer are more likely to lie on the exact Pareto frontier, and thus require more precise active range computation.

(1) The First Layer. In DEG, the first traversed node is assigned an active range of $[0, 1]$ simply because the neighbor set NS is initially empty, which is clearly inaccurate. More importantly, the active range computation for nodes in the remaining layers depends on the active ranges of nodes in the first layer, making the accuracy of the first-layer active ranges crucial. To address this, AC-Prune introduces a bidirectional two-stage scan over the first-layer candidate neighbors to compute more accurate active ranges. Specifically, in the first-stage scan, AC-Prune traverses the first-layer candidate neighbors in forward order to obtain the most reliable active ranges of the final retained node in $LocalNS$ (line 4–6). After completing the forward scan, we perform a reverse scan with the same pruning operations, further refining the active ranges of the remaining first-layer neighbors (line 7).



(a) The first layer. **(b) The remaining layers.**

Figure 6: Illustration of AC-Prune.

(2) The Remaining Layers. Since the proportion of candidate neighbors in the first layer is relatively small (only 4.53%), the majority of the computational cost during the pruning phase is incurred by computing the active ranges of candidate neighbors in the remaining layers. In Algorithm 2, when computing the active ranges of each neighbor, DEG compares it with all nodes currently retained in NS, which leads to substantial computational overhead. To address it, AC-Prune leverages a Pareto angle threshold $\theta_{th} \in [0, \frac{\pi}{2}]$ (see line 12) to skip distance computations with a subset of nodes that violate the angular constraint, thereby reducing redundant distance computations and improving pruning efficiency.

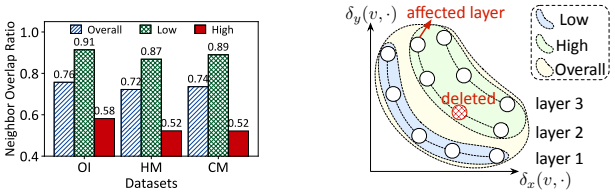
Example. Figure 6 illustrates the pruning process of the candidate neighbor set for node v. For the candidate neighbors in the first layer, AC-Prune traverses them in natural order (e.g., the x-dimension), proceeding from node a to node e, as shown in Figure 6a①. Through this traversal, AC-Prune computes the active ranges of node e and inserts it into the neighbor set NS. Since node e is compared with all other candidates in the first layer during this traversal, its active range is more accurate. Subsequently, AC-Prune traverses the remaining candidates of the first layer in reverse order to compute their active ranges, proceeding from node d back to node a, as illustrated in Figure 6a②. For candidate neighbors in the remaining layers, we consider the first node f in the second layer as an example, as shown in Figure 6b. At this point, nodes a, b, c, and e remain in the current neighbor set NS, while node d has been pruned. AC-Prune compares node f with all nodes in the current neighbor set NS that satisfy the angle constraint. For instance, for node a, AC-Prune computes $|\phi(v, a) - \phi(v, f)|$. If $|\phi(v, a) - \phi(v, f)| \leq \theta_{th}$, AC-Prune computes the pruning range induced by a; otherwise, the computation is skipped.

4.3 Complexity Analysis

Given a target node v with candidate neighbor set C, we analyze the computational complexity of the proposed AC-Prune and the pruning strategy in Algorithm 2.

Baseline Complexity. During pruning, each candidate neighbor is compared against the currently retained nodes, whose size is bounded by the maximum neighbor limit R. Therefore, the total number of distance computations is $T_{\text{baseline}} = O(R \cdot |C|)$.

Complexity of AC-Prune. In contrast, AC-Prune introduces an angle constraint to reduce unnecessary distance computations. When processing the i-th candidate neighbor, AC-Prune compares it against a subset K of the currently retained nodes that satisfy the angle constraint, where $|K| \leq R$. Therefore, the total number of distance computations is $T_{\text{AC-Prune}} = O(|K| \cdot |C|)$.



(a) Neighbor Similarity **(b) Example of an affected node**

Figure 7: Neighbor similarity of affected nodes before and after reinsertion-based repair.

Assuming that the Pareto angles of candidate neighbors are uniformly distributed in $[0, \frac{\pi}{2}]$, the probability that a distance computation is triggered is $P = \frac{4\theta_{th}}{\pi} - \frac{4\theta_{th}^2}{\pi^2}$, where $\theta_{th} \in [0, \frac{\pi}{2}]$ and is set to $\frac{\pi}{18}$ by default. Thus, the expected total complexity becomes $\mathbb{E}[T_{\text{AC-Prune}}] = O(P \cdot R \cdot |C|) = P \cdot T_{\text{baseline}}$.

Discussion. Under the ideal case where the Pareto angles of candidate neighbors are uniformly distributed, AC-Prune reduces the expected number of distance computations by a factor of P. Since θ_{th} is typically small (e.g., $\theta_{th} = \frac{\pi}{18}$), P is generally much smaller than 1, resulting in a significant reduction in distance evaluations. In the worst case, when the Pareto angles are not uniformly distributed, most retained nodes may satisfy the angle constraint (i.e., $P \to 1$). In this situation, AC-Prune degenerates to the baseline pruning strategy with $T_{\text{AC-Prune}} = T_{\text{baseline}}$.

5 CONNECTIVITY-LOSS-DRIVEN INCREMENTAL PARETO FRONTIER REPAIR

As discussed in Section 2.3, applying single-vector repair methods for DVQ indices that incorporate Pareto frontiers is unable to simultaneously ensure update efficiency and retrieval effectiveness. To address this issue, we introduce a *connectivity-loss-driven adaptive repair algorithm* to maintain graph connectivity under deletions while avoiding unnecessary repair operations. We also provide a theoretical analysis of its computational complexity.

5.1 Observation & Intuition

For each affected node, reinsertion-based repair (RBR) methods perform Greedy Pareto Frontier Search from entry nodes to reconstruct its candidate neighbors. However, reinsertion incurs substantial redundant computations, resulting in inefficient updates.

Observation. To investigate the feasibility of incremental repair in Pareto-frontier-based indices under deletions, we measure the out-neighbor overlap ratio of affected nodes on three real-world datasets, as shown in Figure 7a. Specifically, we compute the overall overlap ratio of the out-neighbors for each affected node v, i.e., $\frac{|N'_{\text{out}}(v) \cap N_{\text{out}}(v)|}{|N_{\text{out}}(v)|}$, where $N_{\text{out}}(v)$ and $N'_{\text{out}}(v)$ denote the out-neighbor sets before and after reinsertion, respectively. Furthermore, we analyze the overlap ratio across different layers of Pareto frontiers. Here, affected layers refer to layers that contain deleted nodes (i.e., layer 2). Low layers are defined as all layers below the lowest affected layer (i.e., layer 1), while high layers refer to the affected layers and all layers above them (i.e., layer 2 and layer 3), as illustrated in Figure 7b. We observe that 1) the overall overlap ratio is at least **72%**, indicating that most original neighbors remain effective after repair, and 2) **87%** of neighbors in low layers are preserved, whereas only **52%** in high layers are retained.

Intuition. These observations provide strong support for incremental repair, which avoids the costly global search over all neighbors required by the reinsertion-based method. Observation 1) suggests that most original neighbors of an affected node still belong to its approximate Pareto frontiers, enabling incremental repair during deletions. Furthermore, Observation 2) suggests that neighbor updates should be performed selectively across layers. Specifically, neighbors in low layers remain unchanged, while those in high layers need to be updated.

5.2 Pareto Frontier Repair Based on Connectivity Loss

We further analyze the distribution of deleted neighbors among affected nodes under a 1% deletion batch on the OpenImage index, with a maximum neighbor size of $R = 40$. The results show that the number of deleted neighbors varies significantly across affected nodes (i.e., in-neighbors of deleted nodes), ranging from 1 to 20, indicating that the degree of connectivity loss induced by deletions differs substantially among affected nodes. We next provide a formal definition of connectivity loss.

Definition 2 (Connectivity Loss (CL)). *The connectivity loss of a node v measures the number of its lost out-neighbors, and its degree is defined as*

$$\text{CL}(v) \triangleq \frac{R - |N_{out}(v)|}{R}, \tag{3}$$

where R denotes the maximum number of neighbors and $N_{out}(v)$ denotes the current out-neighbor set of v. It holds that $\text{CL}(v) \in [0, 1]$.

Ideally, each node in the index maintains R out-neighbors, resulting in no connectivity loss (CL = 0). In practice, due to imperfect construction or deletions, nodes may suffer from connectivity loss. To this end, we categorize the degree of connectivity loss into three levels, mild, moderate, and severe. Specifically, for a node v, mild connectivity loss corresponds to $\text{CL}(v) \in [0, \text{CL}_1]$, moderate connectivity loss corresponds to $\text{CL}(v) \in (\text{CL}_1, \text{CL}_2]$, and severe connectivity loss corresponds to $\text{CL}(v) \in (\text{CL}_2, 1]$.

Connectivity-Loss-Driven Adaptive Repair. Based on the above analysis, we argue that the repair algorithm for DVQ indexes should perform incremental repair while following the Approximate Pareto Frontier principle (detailed in Section 2.2) adopted during index construction, i.e., progressively expanding candidate neighbors in a layer-by-layer manner starting from lower layers. To this end, we design a connectivity-loss-driven adaptive Pareto frontier repair algorithm, CA-PFR, to restore the connectivity of the graph index. The core idea of CA-PFR is to determine the repair intensity for affected nodes according to the degree of connectivity loss. The detailed procedure of CA-PFR is shown in Algorithm 4.

Specifically, when a set of nodes $\mathcal{D}$ is deleted, CA-PFR repairs the affected nodes $V_{affected}$, i.e., the in-neighbors of the deleted nodes, according to their degree of connectivity loss. For each affected node $v \in V_{affected}$, CA-PFR first organizes its out-neighbors into multi-layer Pareto frontiers $\mathcal{L} = \{PF_v^1, \ldots, PF_v^l\}$ (line 2), and then obtains its active out-neighbors $Out \leftarrow N_{out}(v) \setminus \mathcal{D}$ (line 3) and active in-neighbors $In \leftarrow N_{in}(v) \setminus \mathcal{D}$ (line 4). Next, the degree of connectivity loss for node v, $\text{CL}(v)$, is computed (line 5) according to Definition 2. Based on $\text{CL}(v)$, CA-PFR adopts historical-Pareto-frontier repair,

Algorithm 4: CA-PFR

Input: DVQ graph $G = (V, E)$, deletion set $\mathcal{D}$, affected nodes $V_{affected}$, out-degree bound R, thresholds $\mathrm{CL}_1, \mathrm{CL}_2$, threshold τ and θ_{th}

Output: Repaired graph on nodes $V' = V \setminus \mathcal{D}$

1 **foreach** $v \in V_{affected}$ **do**
2 $\quad \mathcal{L} \leftarrow \{PF_v^1, \ldots, PF_v^l\}$;
3 $\quad Out \leftarrow N_{out}(v) \setminus \mathcal{D}$;
4 $\quad In \leftarrow N_{in}(v) \setminus \mathcal{D}$;
5 $\quad \mathrm{CL}(v) \leftarrow \frac{R-|Out|}{R}$;
6 $\quad$ **if** $0 < \mathrm{CL}(v) \le \mathrm{CL}_1$ **then** // Mild
7 $\quad\quad C \leftarrow Out \cup In$;
8 $\quad$ **else if** $\mathrm{CL}_1 < \mathrm{CL}(v) \le \mathrm{CL}_2$ **then** // Moderate
9 $\quad\quad \mathcal{L}_A \leftarrow \{PF_v^i \mid PF_v^i \cap \mathcal{D} \neq \emptyset\}$;
10 $\quad\quad Out^{(2)} \leftarrow \bigcup_{PF \in \mathcal{L}_A} \bigcup_{p \in PF} (N_{out}(p) \setminus \mathcal{D})$;
11 $\quad\quad C \leftarrow Out \cup In \cup Out^{(2)}$;
12 $\quad$ **else** // Severe
13 $\quad\quad \ell_{\min} \leftarrow \min\{\, i \mid PF_v^i \cap \mathcal{D} \neq \emptyset \,\}$;
14 $\quad\quad \mathcal{L}_H \leftarrow \{PF_v^i \mid i \ge \ell_{\min}\}$;
15 $\quad\quad Out^{(2)} \leftarrow \bigcup_{PF \in \mathcal{L}_H} \bigcup_{p \in PF} (N_{out}(p) \setminus \mathcal{D})$;
16 $\quad\quad C \leftarrow Out \cup In \cup Out^{(2)}$;
17 $\quad N_{out}(v) \leftarrow$ AC-Prune$(v, C, R, \tau, \theta_{th})$;

affected-layer repair, and multi-layer repair to handle nodes with mild, moderate, and severe connectivity loss, respectively.

Historical-Pareto-Frontier Repair. The candidate set C consists of v's active out-neighbors Out and active in-neighbors In (line 7). Due to the low connectivity loss, this strategy leverages the in-neighbors of affected nodes to efficiently reduce connectivity degradation with minimal overhead. In particular, it avoids any distance computations during candidate selection, thereby significantly reducing the overall repair cost. The key idea is that, due to the reverse edges introduced during index construction, the in-neighbors of a node have previously served as its candidate out-neighbors.

Affected-Layer Repair. The candidate set C consists of Out, In, and an additional expanded set $Out^{(2)}$ derived from the affected layers of Pareto frontiers (i.e., layer 2 in Figure 7b). Specifically, CA-PFR first identifies the affected layers in $\mathcal{L}$ that contain deleted nodes, denoted by $\mathcal{L}_A$ (line 9). It then expands the nodes in these affected layers to form the expanded set $Out^{(2)}$ (line 10). This expansion strategy is motivated by the observation that neighbors in layers below the affected layers are rarely updated during repair. By expanding nodes in these affected layers, CA-PFR can effectively restore local connectivity while avoiding unnecessary exploration.

Multi-Layer Repair. The candidate set C consists of Out, In, and an expanded set $Out^{(2)}$ derived from all high layers (i.e., the "High" region in Figure 7b). Specifically, CA-PFR first identifies the first affected layer containing deleted nodes, denoted by $\ell_{\min}$ (line 13). It then collects all layers of Pareto frontiers whose layer IDs are not smaller than $\ell_{\min}$, denoted by $\mathcal{L}_H$ (line 14), and gathers the active out-neighbors of nodes in these layers to form $Out^{(2)}$ (line 15). This expansion strategy is designed not only to address significant connectivity loss caused by deletions, but also to mitigate connectivity deficiencies inherited from the index construction phase, thereby further improving overall graph connectivity. For example, in the

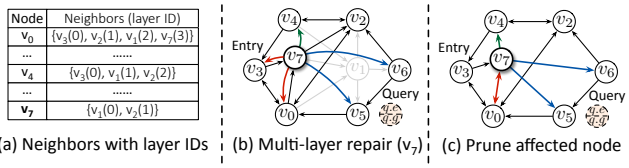


Figure 8: An illustrative example of multi-layer repair.

LAION dataset, approximately 2.77% of nodes in the initial index exhibit out-degrees below CL_2 before deletions. After 30 batches of deletions, the graph connectivity does not degrade; instead, the number of nodes with out-degree below CL_2 decreases by 3.82×.

Finally, the candidate set C is pruned to obtain the repaired out-neighbors $N_{out}(v)$ (line 17), thereby completing the structural repair of node v in the index.

Example. To better illustrate how CA-PFR adaptively repairs the in-neighbors affected by a deleted node, we focus on the case of severe connectivity loss, corresponding to node v_7, as shown in Figure 8. For the affected node v_7, CA-PFR first computes its connectivity loss $\mathrm{CL}(v_7) = 3/4$. Since $\mathrm{CL}(v_7)$ exceeds CL_2 (with the default setting given in Section 6), CA-PFR classifies v_7 as exhibiting severe connectivity loss and thus triggers the multi-layer repair strategy. Specifically, CA-PFR first augments its candidate set by incorporating its in-neighbors (i.e., nodes v_0 and v_3). CA-PFR then identifies all nodes in the high layers (i.e., layer 0 containing the deleted node v_1 and its higher layer 1 containing node v_2) and collects all active out-neighbors of these nodes to construct the final candidate set, i.e., $\{v_0, v_2, v_3, v_4, v_5, v_6\}$. Finally, node v_7 connects to v_0, v_4, v_5 and v_6 after the pruning operation, as shown in Figure 8c.

Notably, this strategy compensates for inherent connectivity deficiencies in the initial index. For example, in the initial graph index (see Figure 2a), reaching v_4 from v_3 follows the path $v_3 \rightarrow v_7 \rightarrow v_2 \rightarrow v_4$ (3 hops). In contrast, after multi-layer repair (see Figure 8c), v_3 can reach v_4 via v_7 in only 2 hops, thereby reducing memory accesses and decreasing search latency.

Asynchronous Reverse Graph Maintenance. During deletion, ACTIVE requires in-neighbor information to identify and repair affected nodes. Since the graph-based index stores only out-neighbors, obtaining in-neighbors would otherwise require scanning the entire graph topology G_{out}. To avoid this overhead, ACTIVE maintains a lightweight reverse graph topology G_{in}, where for each node v, $N_{\text{out}}^{G_{in}}(v) = N_{\text{in}}^{G_{out}}(v)$, thereby enabling $O(1)$ access to in-neighbor information. Unlike G_{out}, whose edges store dual-vector distances and active ranges, G_{in} stores only in-neighbor IDs, accounting for merely 2.34% of the combined size of G_{out} and the dual-vector data in the HM dataset. Since batch updates modify edges in G_{out}, ACTIVE asynchronously maintains G_{in} in the background to minimize synchronization overhead.

5.3 Computational Overhead Analysis

Given a deleted node, CA-PFR first adaptively augments the candidate set C for each affected node $v \in V_{affected}$, resulting in a updated candidate set C', with computational complexity $T_{acs} = O(|\Delta C|)$, where ΔC denotes the newly introduced candidate neighbors of node v. Subsequently, the updated candidate set C' is pruned to obtain the final neighbors of v, leading to a computational complexity of $T_{pruning} = O(R \cdot |C'|)$, where R denotes the maximum out-degree and $|C'| = |C| + |\Delta C| \approx R + |\Delta C|$ due to $|C| \approx R$.

Next, we analyze the size of $|\Delta C|$ under the three repair strategies of CA-PFR, namely historical-Pareto-frontier, affected-layer, and multi-layer repair.

Historical-Pareto-Frontier Repair (Mild). For each affected node $v \in V_{affected}$, this strategy augments the original active out-neighbors in C with the active in-neighbors of v, resulting in $|\Delta C| \approx |N_{in}(v)|$. Since the dual-vector distances $\delta_x(v, v_{in})$ and $\delta_y(v, v_{in})$ are already stored in the corresponding in-neighbors $v_{in} \in N_{in}(v)$, the candidate set augmentation incurs no additional distance computations, i.e., $T_{acs} = 0$. Therefore, the computational complexity of repairing each affected node v is: $T_{mild} = T_{pruning} \approx O\left(R^2 + R \cdot |N_{in}(v)|\right)$.

Affected-Layer Repair (Moderate). In addition to the active in-neighbors $N_{in}(v)$, this repair strategy further adds the active out-neighbors of nodes in the affected layers to C', yielding $|\Delta C| \approx |N_{in}(v)| + n \cdot R$, where n is the average number of nodes per layer, and $n \leq R$. Thus, the complexity of repairing each affected node v is: $T_{moderate} = T_{acs} + T_{pruning} = O\left((n+1) \cdot R^2 + R \cdot |N_{in}(v)| + n \cdot R + |N_{in}(v)|\right)$.

Multi-Layer Repair (Severe). This repair strategy not only includes the in-neighbors and the original active out-neighbors, but also incorporates all active out-neighbors of nodes in the high layers into C', resulting in $|\Delta C| \approx |N_{in}(v)| + k \cdot n \cdot R$, where k is the number of the high layers, and $k \cdot n \leq R$. Therefore, the computational complexity of repairing each affected node v is: $T_{severe} = T_{acs} + T_{pruning} = O\left((kn+1) \cdot R^2 + R \cdot |N_{in}(v)| + k \cdot n \cdot R + |N_{in}(v)|\right)$.

Let p_1, p_2, and p_3 denote the probabilities of triggering the three repair strategies, respectively, satisfying $p_1 + p_2 + p_3 = 1$. As a result, the computational complexity of CA-PFR is: $T_{CA-PFR} = p_1 \cdot T_{mild} + p_2 \cdot T_{moderate} + p_3 \cdot T_{severe}$. In practice, since severe repair is triggered infrequently, with probabilities satisfying $p_1 > p_2 \gg p_3$, the expected complexity can be simplified as: $\mathbb{E}[T_{CA-PFR}] = p_1' \cdot T_{mild} + p_2' \cdot T_{moderate}$, where $p_1' = \frac{p_1}{p_1+p_2}, p_2' = \frac{p_2}{p_1+p_2}$.

6 EVALUATION

6.1 Experimental Setup

Evaluation Platform. All experiments are conducted on a server with an Intel(R) Xeon(R) Gold 6248R CPU, 320 GB memory. The system runs Ubuntu 22.04 LTS with GCC version 11.4.0.

Datasets. We evaluate our system on four publicly available real-world datasets, namely OpenImage (OI) [58], Howto100M (HM) [58], CC3M (CM) [58], and LAION (LN) [41]. The details of these datasets are listed in Table 1.

Compared Methods We comprehensively evaluate ACTIVE against baselines in both static search and dynamic updates to validate its search and update performance.

Static Search We select multi-index retrieval (MR) [45, 60] and DEG [58] as baselines for static search. MR builds a separate single-vector index (i.e., an HNSW index) for each of the two attribute vectors. DEG constructs an α-agnostic unified index for the two attribute vectors and selects approximate Pareto frontiers as candidate neighbors. The key difference between ACTIVE and DEG in terms of construction lies in their active range computation approach, as detailed in Section 4.2.

Table 1: Dataset statistics. D_x and D_y denote the vector dimensions of the x- and y-modalities, respectively. Each dataset contains 1,000 query vectors.

Dataset	D_x	D_y	#Vectors	Contents
OpenImage (OI)	768	768	507,444	Text & Image
Howto100M (HM)	1024	768	1,238,875	Text & Video
CC3M (CM)	768	768	3,131,153	Text & Image
LAION (LN)	512	512	10,642,155	Text & Image

Table 2: Construction time comparison on different datasets.

Dataset	MR	DEG	ACTIVE
OpenImage	565.42	931.58	726.66
Howto100M	1381.85	2250.63	1805.45
CC3M	3777.26	5143.36	4214.39
LAION	13893.70	11517.60	10055.28

Dynamic Updates Since no dedicated methods have been proposed for handling updates in dynamic DVQ indexes, we adopt representative RBR [57] and NRBR [44] methods from single-vector graph indexes as baselines for deletions, and the incremental construction strategy of DEG for insertions. These baselines employ existing pruning algorithms [58] for active range computation.

Parameter Settings During index construction, the search candidate set size $ef_{construct}$ is set to 200, and the maximum number of edges per node R is 40. The Pareto angle threshold parameter θ_{th} in ACTIVE is set to $\pi/18$. During index updates, the search candidate set size ef_{update} is set to 100, which is used for reinsertions in RBR as well as for insertions in both ACTIVE and DEG. The parameters CL_1 and CL_2 are set to 1/3 and 1/2, respectively. During query processing in dynamic updates, the search candidate set size ef_{search} is set to 100. Unless otherwise specified, all update operations are performed using 32 threads, and the α value of each query is randomly sampled from the range $[0, 1]$.

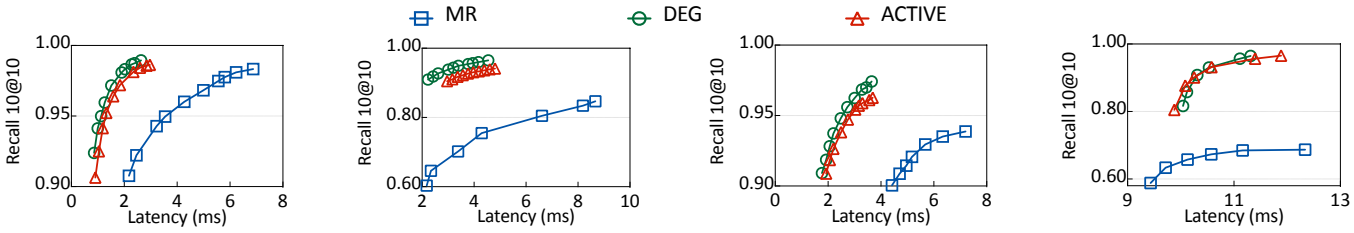
Workloads. Since no existing unified DVQ index supports both deletions and insertions for dual vectors, we conduct separate evaluations for deletions and insertions. We construct a static DVQ index on 100% of each dataset for static search and deletion experiments, and another index on 50% of each dataset for insertion experiments.

For static search, we use the index built on 100% of each dataset to evaluate search latency (in milliseconds) and Recall@10, with the search candidate set size ef_{search} varying from 10 to 200. For deletions, we use the same 100% index and remove 1% of the vectors in each of 10 batches, resulting in a total of 10% deletions. For insertions, we use the 50% index and insert 1% of the vectors in each of 10 batches, resulting in a total of 10% insertions.

Metrics. We measure search accuracy using Recall@10 and search efficiency using average query latency, while update (i.e., deletions and insertions) performance is evaluated using throughput (deletions/insertions per second) and computational overhead is measured by the number of distance computations during updates.

6.2 Static Index Quality

The accuracy-efficiency trade-off results of ACTIVE and the two baselines on the four datasets are shown in Figure 9. The results demonstrate that Pareto-frontier-based indices consistently achieve



(a) OpenImage (b) Howto100M (c) CC3M (d) LAION

Figure 9: Comparison of the trade-offs between search accuracy and latency. The top left is better.

better performance than preference-specific multi-indices. For example, on the LN dataset, ACTIVE improves search accuracy by up to 28.31% over MR under the same search latency. This indicates that preference-specific multi-index approaches such as MR cannot maintain high retrieval accuracy under arbitrary α values, making them unsuitable for the Dual-Vector Query problem. Furthermore, ACTIVE achieves an average 1.22× reduction in construction time compared to DEG while maintaining a similar accuracy-efficiency trade-off curve, as shown in Table 2. These improvements are largely attributed to our proposed angle-constrained dynamic pruning algorithm (AC-Prune).

6.3 Update Performance

We evaluate the update (i.e., deletion and insertion) performance of different methods on four real-world vector datasets. Due to randomness in index construction, we repeat each experiment three times and report the average results.

Deletion Throughput Comparison. As shown in Figure 10, ACTIVE consistently achieves higher throughput than both RBR and NRBR across all datasets, improving deletion throughput by 22.64–30.28× over RBR and 1.64–2.06× over NRBR. The key advantages of ACTIVE lie in its connectivity-loss-driven adaptive repair (CA-PFR) strategy and the angle-constrained pruning (AC-Prune) algorithm, both of which substantially reduce the computational overhead during deletions. It is noteworthy that the throughput of ACTIVE is relatively low in the first batch, since the reverse graph needs to be explicitly initialized, whereas it can be maintained asynchronously in subsequent batches.

Insertion Throughput Comparison. Under the 1% insertion workload, ACTIVE improves insertion throughput by 1.26–1.43× compared with the incremental construction of DEG, as shown in Figure 11. The improvement is attributed to AC-Prune, which reduces the number of distance computations by introducing Pareto-angle constraints during pruning.

Distance Computation Comparison. We further compare the number of distance computations of ACTIVE against other methods across all datasets, as shown in Figure 12. Under 1% deletions, ACTIVE consistently requires fewer computations than RBR and NRBR, reducing the computation count by 8.72×–21.46× over RBR and by 1.76×–2.53× over NRBR. This is attributed to the fact that CA-PFR adaptively selects different repair strategies for each affected node instead of applying a fixed repair strategy, even when the connectivity loss CL of the affected node is small. Under 1% insertions, ACTIVE reduces the number of computations compared to DEG, achieving a reduction of 1.34×–1.43×, thanks to AC-Prune.

6.4 Index Quality

Although several designs in ACTIVE , including connectivity-loss-driven adaptive repair and the angle-constrained dynamic pruning algorithm, substantially improve update performance, it remains necessary to evaluate whether ACTIVE can maintain graph index quality during updates. To this end, we measure search accuracy and average search latency (ms) of different methods under 1% deletions and 1% insertions.

Search Accuracy Comparison under deletions. As shown in Figure 13, we report the search Recall@10 of different deletion methods under 1% deletions, with $ef_{\text{search}} = 100$. Compared with RBR, ACTIVE achieves comparable recall across most datasets, indicating that our local adaptive repair strategy can closely match the effectiveness of global reconstruction for affected nodes. On the HM dataset, although RBR achieves higher search accuracy than ACTIVE , it comes at the cost of a substantially higher deletion overhead, with 30.28× slower deletion throughput. Compared with NRBR, ACTIVE improves search accuracy by 0.21%–1.81%, demonstrating that our connectivity-loss-driven adaptive repair strategy is more suitable for repairing Pareto frontiers than the simple neighbor-reconnection repair strategy, such as FreshDiskANN.

Search Accuracy Comparison under Insertions. We compare the search accuracy of ACTIVE with DEG under 1% insertions across all datasets, with $ef_{\text{search}} = 100$. Figure 14 shows that ACTIVE achieves recall comparable to that of DEG across all datasets, indicating that our proposed AC-Prune improves pruning efficiency while maintaining search accuracy closely matching the baseline.

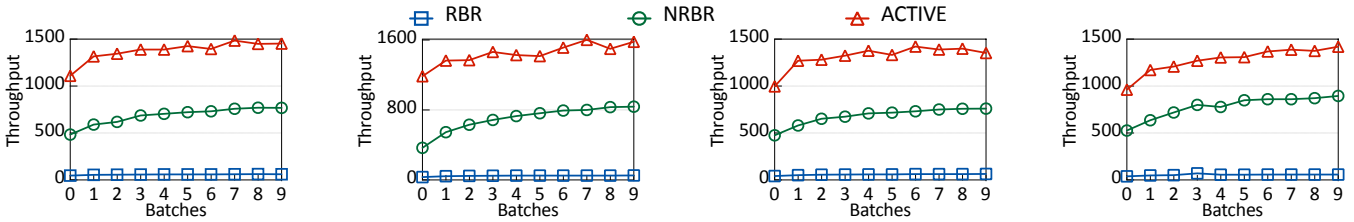
Search Latency Comparison. To further assess the impact of different update methods on search performance, we measure the average search latency on the largest dataset, LN, separately under deletions and insertions. As shown in Figure 15, ACTIVE achieves search latency comparable to that of other methods and remains consistently stable under both deletions and insertions.

These results suggest that the AC-Prune and CA-PFR components in ACTIVE improve update performance while preserving navigability and stability, while also maintaining higher index quality than the baselines.

6.5 Scalability

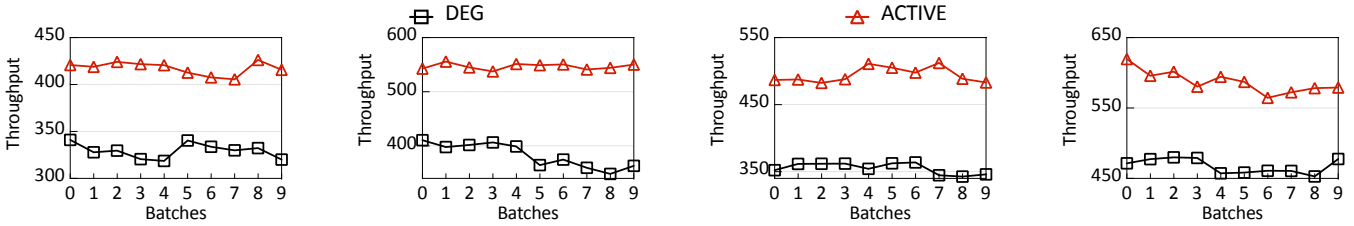
We evaluate deletion and insertion performance on the OI dataset under different batch sizes to assess scalability with respect to update granularity.

Throughput under Different Batch Sizes. As shown in Figure 16a and 16b, When the batch size increases from 0.1% to 10% of the dataset, all methods achieve higher update throughput. This is because larger batch updates amortize fixed system overheads,



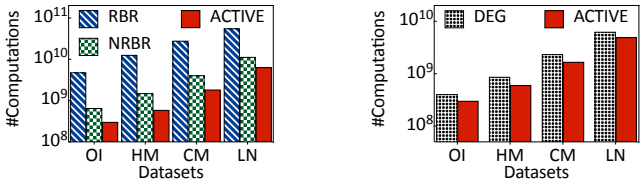
(a) OpenImage (b) Howto100M (c) CC3M (d) LAION

Figure 10: Throughput comparison under 1% deletions.



(a) OpenImage (b) Howto100M (c) CC3M (d) LAION

Figure 11: Throughput comparison under 1% insertions.



(a) 1% Deletion (b) 1% Insertion

Figure 12: Distance computation comparison.

thereby improving overall throughput. Notably, ACTIVE consistently outperforms all baselines across different batch sizes under both deletions and insertions, indicating that our proposed AC-Prune and CA-PFR provide a consistent performance advantage across a wide range of update granularity.

Search Accuracy under Different Batch Sizes. As shown in Figure 16c, as the batch size increases, the recall of both ACTIVE and RBR consistently improves, while NRBR degrades. This indicates that NRBR cannot effectively compensate for connectivity loss under large-batch deletions. In contrast, ACTIVE consistently achieves recall comparable to RBR across all batch sizes. This benefit stems from our proposed CA-PFR, which adaptively selects different repair strategies based on the connectivity loss of affected nodes, thereby maintaining the connectivity of the graph index even under large-batch deletions. In Figure 16d, ACTIVE consistently achieves search accuracy no worse than DEG across all batch sizes (0.1%, 1%, and 10%). This demonstrates that our proposed AC-Prune is effective across different insertion batch sizes, ensuring that ACTIVE improves insertion (incremental construction) throughput while maintaining search accuracy comparable to DEG.

6.6 Parameter Sensitivity

In ACTIVE , update performance is mainly affected by three parameters, namely the Pareto angle threshold θ_{th} in AC-Prune, and the connectivity loss thresholds CL_1 and CL_2 in CA-PFR. We next study the impact of these parameters on update throughput and recall by varying them, and motivate the choice of the default settings.

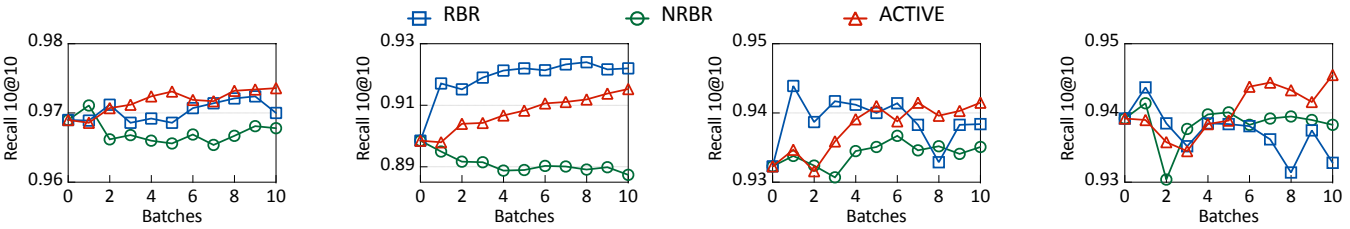
Pareto Angle Threshold. In AC-Prune, θ_{th} defines the angle constraint threshold, where a larger θ_{th} allows each candidate node to be compared with more previously retained nodes, resulting in lower pruning efficiency. To evaluate its impact, we conduct a sensitivity analysis on three datasets under 1% deletions and insertions by varying θ_{th} among $\pi/18$, $\pi/6$, $5\pi/18$, $7\pi/18$, and $\pi/2$, as shown in Figure 17. The results show that $\pi/18$ achieves consistently better throughput and recall than the other settings. Therefore, we set θ_{th} to $\pi/18$ by default.

Connectivity Loss Thresholds. In CA-PFR, CL_1 and CL_2 define the boundaries between mild and moderate connectivity loss, and between moderate and severe connectivity loss, respectively. As CL_1 and CL_2 increase, more affected nodes will be handled by more lightweight and localized repair strategies. To this end, we conduct a sensitivity analysis on the OI dataset under 1% deletions and insertions, as shown in Figure 18. The results show that the setting $CL_1 = 1/3$ and $CL_2 = 1/2$ achieves the best overall performance in both throughput and recall.

6.7 Performance Gain

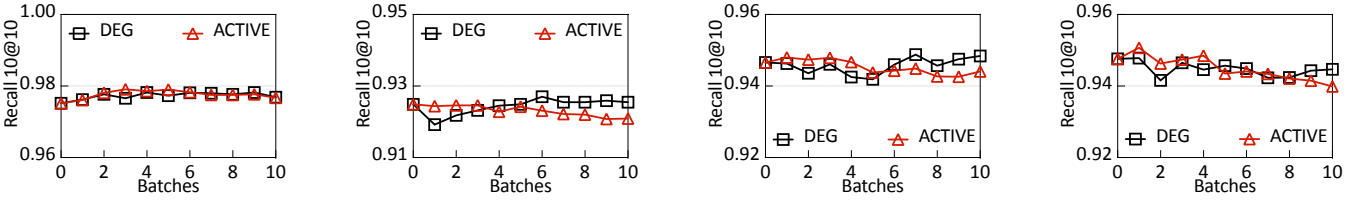
Since ACTIVE differs from DEG only in the pruning strategy during insertion, the performance gains can be attributed to our proposed AC-Prune, as shown in Figure 11. To evaluate the contribution of each core component in ACTIVE to deletion performance, we start from the baseline method RBR and progressively integrate the proposed components. We conduct the evaluation on three datasets under 0.1%, 1%, and 10% deletion batch sizes, as shown in Figure 19, where deletion performance is measured by normalized throughput relative to the baseline. The results show that each component consistently contributes to substantial deletion speedup across different batch sizes.

Specifically, we first introduce the connectivity-loss-driven adaptive repair algorithm (+C.A.), which adaptively repairs affected nodes according to the degree of connectivity loss, achieving average speedups of 13.85×, 11.52×, and 11.34× under the three batch sizes, respectively. We then further incorporate an asynchronously maintained reverse graph (+Async.), increasing the



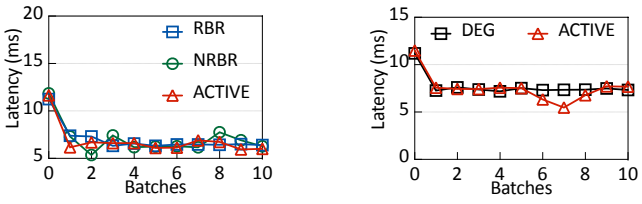
(a) OpenImage (b) Howto100M (c) CC3M (d) LAION

Figure 13: Search accuracy comparison under 1% deletions.



(a) OpenImage (b) Howto100M (c) CC3M (d) LAION

Figure 14: Search accuracy comparison under 1% insertions.



(a) 1% Deletion (b) 1% Insertion

Figure 15: Search latency comparison.

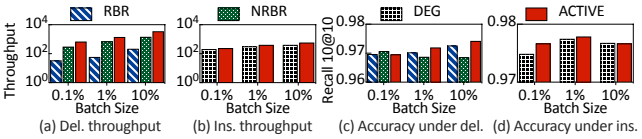
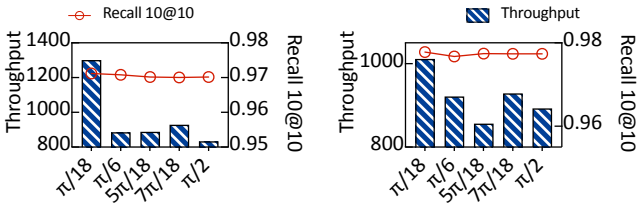
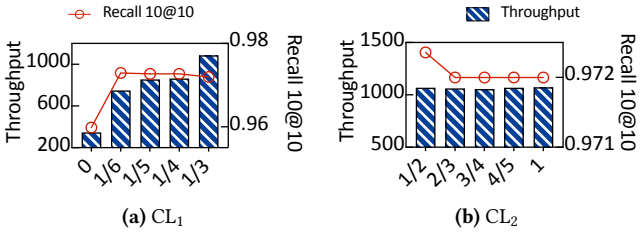


Figure 16: Varying batch size of deletions and insertions.



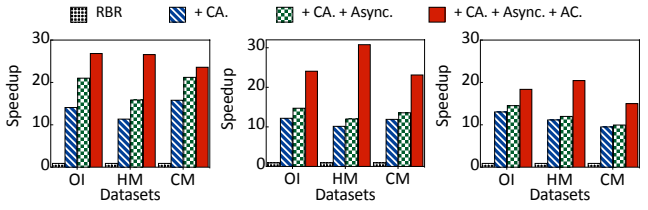
(a) 1% Deletion (b) 1% Insertion

Figure 17: Sensitivity to the Pareto angle threshold θ_{th}.



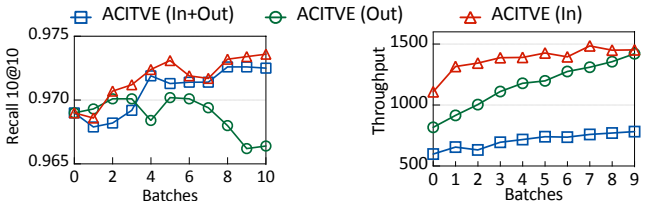
(a) CL_1 (b) CL_2

Figure 18: Sensitivity to connectivity loss thresholds.



(a) 0.1% Deletion (b) 1% Deletion (c) 10% Deletion

Figure 19: Performance gain.



(a) Search accuracy (b) Deletion Throughput

Figure 20: Impact of different affected node definitions.

average speedups to 19.46×, 13.54×, and 12.27×, respectively, by avoiding the need to scan the entire graph topology. Finally, we introduce the angle-constrained pruning algorithm (+A.C.), which reduces comparisons between candidate nodes and previously retained nodes through angle-based filtering, further improving the average speedups to 25.76×, 26.11×, and 18.05×, respectively. These results demonstrate that all components in ACTIVE consistently contribute to improving deletion performance.

Notably, the asynchronously maintained reverse graph provides significant performance gains under the 0.1% batch size. Its main benefit lies in eliminating full graph scans, whereas the cost of in-memory access is relatively small. Therefore, this design is most effective when only a very small fraction of nodes are affected, which is precisely the case in the 0.1% deletion setting. For example, under the 0.1% batch size, only 5.63% of nodes in the OI dataset are affected.

6.8 Analysis of Affected Nodes

By default, ACTIVE treats in-neighbors of deleted nodes as affected nodes, since deletions mainly cause connectivity loss of in-neighbors, thereby degrading graph connectivity. We argue that repairing only in-neighbors is sufficient to largely restore connectivity. To validate this, we extend the repair strategy in ACTIVE (In) to two variants: ACTIVE (Out) and ACTIVE (In+Out), which respectively repair only out-neighbors and both in- and out-neighbors

of deleted nodes. For out-neighbor repair, we follow the historical Pareto-frontier repair principle, where an out-neighbor may have previously served as an in-neighbor. As shown in Fig. 20, ACTIVE (In) consistently achieves higher update throughput and better search accuracy than other repair strategies.

7 RELATED WORK

Hybrid Search. To meet the demands of increasingly complex real-world applications, various hybrid search paradigms have been proposed, including filtered ANNS [32, 42, 51, 61] and multi-vector ANNS [45]. Filtered ANNS associates each object with both vector representations and structured attributes, supporting approximate nearest neighbor search under attribute constraints such as numerical ranges [12, 23, 25, 33, 50, 55, 63], temporal conditions [49], and categorical filters [6, 16, 53]. Multi-vector ANNS represents each object with multiple embeddings and jointly optimizes similarity across vector spaces [4, 8, 11, 37, 38, 58]. These paradigms extend traditional ANNS by incorporating structured or multi-view semantic information.

Dual-Vector Query. Dual-vector queries are a special case of hybrid search and have been widely studied [13, 14, 45, 58]. Each object is associated with two vectors in different embedding spaces, and queries retrieve objects with the minimum weighted distance determined by a query-specified preference parameter. Existing approaches fall into two categories: preference-specific multi-index methods [45, 60] and preference-agnostic unified index methods [58]. The former fails to maintain high search accuracy under arbitrary query preferences, while the latter achieves a better accuracy–efficiency trade-off but does not support efficient dynamic updates. Overall, there is still no index that simultaneously supports efficient dynamic updates and maintains high-quality retrieval under arbitrary query preferences.

Graph-based Index Updates. In single-vector graph-based indices, dynamic updates [17, 26, 54, 59] have been extensively studied. Insertions are typically performed via incremental construction [26, 44, 59], while deletions are handled by either global reinsertion [28, 57] or local neighbor reconnection [44, 59].

8 CONCLUSION

We propose ACTIVE , a dynamic Pareto-frontier-based unified DVQ index that supports efficient updates while maintaining high retrieval quality under arbitrary query preferences. To accelerate index construction and insertions, we design an angle-constrained dynamic pruning algorithm, AC-Prune, which reduces distance computations via Pareto-angle-based constraints. To support efficient updates, we further propose a connectivity-loss-driven adaptive repair algorithm (CA-PFR) that selects different repair strategies based on the connectivity loss of affected nodes. Extensive experiments demonstrate that ACTIVE consistently outperforms existing baselines in static search, deletions, and insertions, achieving a strong trade-off between retrieval quality and update throughput across diverse workloads.

REFERENCES

[1] 2025. pgvector. https://github.com/pgvector/pgvector.

[2] Akari Asai, Sewon Min, Zexuan Zhong, and Danqi Chen. 2023. Retrieval-based Language Models and Applications. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics: Tutorial Abstracts, ACL 2023, Toronto, Canada, July 9-14, 2023*. Association for Computational Linguistics, 41–46.

[3] Tadas Baltrušaitis, Chaitanya Ahuja, and Louis-Philippe Morency. 2018. Multimodal machine learning: A survey and taxonomy. *IEEE transactions on pattern analysis and machine intelligence* 41, 2 (2018), 423–443.

[4] Zheng Bian, Man Lung Yiu, and Bo Tang. 2025. IGP: Efficient Multi-Vector Retrieval via Proximity Graph Index. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Padua, Italy) *(SIGIR '25)*. Association for Computing Machinery, New York, NY, USA, 2524–2533. https://doi.org/10.1145/3726302.3730004

[5] S. Borzsony, D. Kossmann, and K. Stocker. 2001. The Skyline operator. In *Proceedings 17th International Conference on Data Engineering*. 421–430. https://doi.org/10.1109/ICDE.2001.914855

[6] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 6, Article 246 (Dec. 2024), 27 pages. https://doi.org/10.1145/3698822

[7] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*. 5199–5212.

[8] Yaoqi Chen, Ruicheng Zheng, Qi Chen, Shuotao Xu, Qianxi Zhang, Xue Wu, Weihao Han, Hua Yuan, Mingqin Li, Yujing Wang, Jason Li, Fan Yang, Hao Sun, Weiwei Deng, Feng Sun, Qi Zhang, and Mao Yang. 2024. OneSparse: A Unified System for Multi-index Vector Search. In *Companion Proceedings of the ACM Web Conference 2024* (Singapore, Singapore) *(WWW '24)*. Association for Computing Machinery, New York, NY, USA, 393–402. https://doi.org/10.1145/3589335.3648338

[9] Zhida Chen, Lisi Chen, Gao Cong, and Christian S. Jensen. 2021. Location- and Keyword-Based Querying of Geo-Textual Data: A Survey. *The VLDB Journal* 30, 4 (2021), 603–640.

[10] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems, Boston, MA, USA, September 15-19, 2016*. ACM, 191–198.

[11] Laxman Dhulipala, Majid Hadian, Rajesh Jayaram, Jason Lee, and Vahab Mirrokni. 2024. MUVERA: Multi-Vector Retrieval via Fixed Dimensional Encodings. arXiv:2405.19504 [cs.DS] https://arxiv.org/abs/2405.19504

[12] Joshua Engels, Benjamin Landrum, Shangdi Yu, Laxman Dhulipala, and Julian Shun. 2024. Approximate Nearest Neighbor Search with Window Filters. arXiv:2402.00943 [cs.DS] https://arxiv.org/abs/2402.00943

[13] Ronald Fagin, Amnon Lotem, and Moni Naor. 2001. Optimal aggregation algorithms for middleware. In *Proceedings of the Twentieth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems* (Santa Barbara, California, USA) *(PODS '01)*. Association for Computing Machinery, New York, NY, USA, 102–113. https://doi.org/10.1145/375551.375567

[14] Maximilian Franzke, Tobias Emrich, Andreas Züfle, and Matthias Renz. 2016. Indexing multi-metric data. In *2016 IEEE 32nd International Conference on Data Engineering (ICDE)*. IEEE, 1122–1133.

[15] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proc. VLDB Endow.* 12, 5 (2019), 461–474.

[16] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference 2023, WWW 2023, Austin, TX, USA, 30 April 2023 - 4 May 2023*. ACM, 3406–3416.

[17] Hao Guo and Youyou Lu. 2026. OdinANN: Direct Insert for Consistently Stable Performance in Billion-Scale Graph-Based Vector Search. In *24th USENIX Conference on File and Storage Technologies (FAST 26)*. USENIX Association, Santa Clara, CA, 133–147. https://www.usenix.org/conference/fast26/presentation/guo

[18] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. 2015. Query-Aware Locality-Sensitive Hashing for Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 9, 1 (2015), 1–12.

[19] H. V. Jagadish, Beng Chin Ooi, Kian-Lee Tan, Cui Yu, and Rui Zhang. 2005. iDistance: An adaptive B+-tree based indexing method for nearest neighbor search. *ACM Trans. Database Syst.* 30, 2 (June 2005), 364–397. https://doi.org/10.1145/1071610.1071612

[20] H. V. Jagadish, Beng Chin Ooi, Kian-Lee Tan, Cui Yu, and Rui Zhang. 2005. iDistance: An adaptive B+-tree based indexing method for nearest neighbor search. *ACM Trans. Database Syst.* 30, 2 (June 2005), 364–397. https://doi.org/10.1145/1071610.1071612

[21] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. Diskann: Fast accurate billion-point

nearest neighbor search on a single node. *Advances in neural information processing Systems* 32 (2019).

[22] Hervé Jégou, Romain Tavenard, Matthijs Douze, and Laurent Amsaleg. 2011. Searching in one billion vectors: Re-rank with source coding. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing, ICASSP 2011, May 22-27, 2011, Prague Congress Center, Prague, Czech Republic*. IEEE, 861–864.

[23] Mengxu Jiang, Zhi Yang, Fangyuan Zhang, Guanhao Hou, Jieming Shi, Wenchao Zhou, Feifei Li, and Sibo Wang. 2025. DIGRA: A Dynamic Graph Indexing for Approximate Nearest Neighbor Search with Range Filter. *Proc. ACM Manag. Data* 3, 3, Article 148 (June 2025), 26 pages. https://doi.org/10.1145/3725399

[24] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data — Experiments, Analyses, and Improvement . *IEEE Transactions on Knowledge & Data Engineering* 32, 08 (Aug. 2020), 1475–1488. https://doi.org/10.1109/TKDE.2019.2909204

[25] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *Proc. VLDB Endow.* 18, 4 (Dec. 2024), 1118–1130. https://doi.org/10.14778/3717755.3717770

[26] Dawei Liu, Bolong Zheng, Ziyang Yue, Fuhao Ruan, Xiaofang Zhou, and Christian S. Jensen. 2025. Wolverine: Highly Efficient Monotonic Search Path Repair for Graph-Based ANN Index Updates. *Proc. VLDB Endow.* 18, 7 (March 2025), 2268–2280. https://doi.org/10.14778/3734839.3734860

[27] Pingchuan Ma, Tao Du, and Wojciech Matusik. 2020. Efficient continuous pareto exploration in multi-task learning. In *Proceedings of the 37th International Conference on Machine Learning (ICML'20)*. JMLR.org, Article 605, 10 pages.

[28] Yury A. Malkov and Dmitry A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (2020), 824–836.

[29] Antoine Miech, Dimitri Zhukov, Jean-Baptiste Alayrac, Makarand Tapaswi, Ivan Laptev, and Josef Sivic. 2019. HowTo100M: Learning a Text-Video Embedding by Watching Hundred Million Narrated Video Clips. In *ICCV*.

[30] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Ali Mousavi, Ihab F. Ilyas, Umar Farooq Minhas, Jeffrey Pound, and Theodoros Rekatsinas. 2023. High-Throughput Vector Similarity Search in Knowledge Graphs. *Proc. ACM Manag. Data* 1, 2 (2023), 197:1–197:25.

[31] Shumpei Okura, Yukihiro Tagami, Shingo Ono, and Akira Tajima. 2017. Embedding-based News Recommendation for Millions of Users. In *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017*. ACM, 1933–1942.

[32] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proc. ACM Manag. Data* 2, 3 (2024), 120.

[33] Zhencan Peng, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proc. VLDB Endow.* 18, 10 (June 2025), 3256–3268. https://doi.org/10.14778/3748191.3748193

[34] Jordi Pont-Tuset, Jasper Uijlings, Soravit Changpinyo, Radu Soricut, and Vittorio Ferrari. 2020. Connecting Vision and Language with Localized Narratives. In *ECCV*.

[35] Jordi Pont-Tuset, Jasper Uijlings, Soravit Changpinyo, Radu Soricut, and Vittorio Ferrari. 2020. Connecting Vision and Language with Localized Narratives. In *ECCV*.

[36] Amaia Salvador, Nicholas Hynes, Yusuf Aytar, Javier Marin, Ferda Ofli, Ingmar Weber, and Antonio Torralba. 2017. Learning cross-modal embeddings for cooking recipes and food images. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 3020–3028.

[37] Keshav Santhanam, Omar Khattab, Christopher Potts, and Matei Zaharia. 2022. PLAID: An Efficient Engine for Late Interaction Retrieval. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management* (Atlanta, GA, USA) *(CIKM '22)*. Association for Computing Machinery, New York, NY, USA, 1747–1756. https://doi.org/10.1145/3511808.3557325

[38] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2021. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. *CoRR* abs/2112.01488 (2021). arXiv:2112.01488 https://arxiv.org/abs/2112.01488

[39] Badrul Munir Sarwar, George Karypis, Joseph A. Konstan, and John Riedl. 2001. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the Tenth International World Wide Web Conference, WWW 10, Hong Kong, China, May 1-5, 2001*. ACM, 285–295.

[40] J. Ben Schafer, Dan Frankowski, Jonathan L. Herlocker, and Shilad Sen. 2007. Collaborative Filtering Recommender Systems. In *The Adaptive Web, Methods and Strategies of Web Personalization (Lecture Notes in Computer Science)*, Vol. 4321. 291–324.

[41] Christoph Schuhmann, Romain Beaumont, Cade W Gordon, Ross Wightman, Theo Coombes, Aarush Katta, Clayton Mullis, Patrick Schramowski, Srivatsa R Kundurthy, Katherine Crowson, et al. 2022. LAION-5B: An open large-scale dataset for training next generation image-text models. (2022).

[42] Gaurav Sehgal and Semih Salihoğlu. 2025. NaviX: A Native Vector Index Design for Graph DBMSs With Robust Predicate-Agnostic Search Performance. *Proc. VLDB Endow.* 18, 11 (July 2025), 4438–4450. https://doi.org/10.14778/3749646.3749704

[43] Piyush Sharma, Nan Ding, Sebastian Goodman, and Radu Soricut. 2018. Conceptual Captions: A Cleaned, Hypernymed, Image Alt-text Dataset For Automatic Image Captioning. In *Proceedings of ACL*.

[44] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. *CoRR* abs/2105.09613 (2021).

[45] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *SIGMOD '21: International Conference on Management of Data, Virtual Event, China, June 20-25, 2021*. 2614–2627.

[46] Mengzhao Wang, Xiangyu Ke, Xiaoliang Xu, Lu Chen, Yunjun Gao, Pinpin Huang, and Runkai Zhu. 2024. Must: An effective and scalable framework for multimodal search of target modality. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 4747–4759.

[47] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *CoRR* abs/2401.02116 (2024).

[48] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *Proc. VLDB Endow.* 14, 11 (July 2021), 1964–1978. https://doi.org/10.14778/3476249.3476255

[49] Yuxiang Wang, Ziyuan He, Yongxin Tong, Zimu Zhou, and Yiman Zhong. 2025. Timestamp Approximate Nearest Neighbor Search Over High-Dimensional Vector Data. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. 3043–3055. https://doi.org/10.1109/ICDE65448.2025.00228

[50] Ziqi Wang, Jingzhe Zhang, and Wei Hu. 2025. WoW: A Window-to-Window Incremental Index for Range-Filtering Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 3, 6, Article 378 (Dec. 2025), 27 pages. https://doi.org/10.1145/3769843

[51] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: a hybrid analytical engine towards query fusion for structured and unstructured data. *Proc. VLDB Endow.* 13, 12 (Aug. 2020), 3152–3165. https://doi.org/10.14778/3415478.3415541

[52] Kyle Williams, Lichi Li, Madian Khabsa, Jian Wu, Patrick C. Shih, and C. Lee Giles. 2014. A Web Service for Scholarly Big Data Information Extraction. In *2014 IEEE International Conference on Web Services, ICWS, 2014, Anchorage, AK, USA, June 27 - July 2, 2014*. IEEE Computer Society, 105–112.

[53] Jingyi Xi, Chenghao Mo, Ben Karsin, Artem Chirkin, Mingqin Li, and Minjia Zhang. 2025. VecFlow: A High-Performance Vector Data Management System for Filtered-Search on GPUs. *Proc. ACM Manag. Data* 3, 4, Article 271 (Sept. 2025), 27 pages. https://doi.org/10.1145/3749189

[54] Haike Xu, Magdalen Dobson Manohar, Philip A. Bernstein, Badrish Chandramouli, Richard Wen, and Harsha Vardhan Simhadri. 2025. In-Place Updates of a Graph Index for Streaming Approximate Nearest Neighbor Search. *CoRR* abs/2502.13826 (2025).

[55] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. 2024. iRangeGraph: Improvising Range-dedicated Graphs for Range-filtering Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 6, Article 239 (Dec. 2024), 26 pages. https://doi.org/10.1145/3698814

[56] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*. ACM, 545–561.

[57] Zhaozhuo Xu, Weijie Zhao, Shulong Tan, Zhixin Zhou, and Ping Li. 2022. Proximity Graph Maintenance for Fast Online Nearest Neighbor Search. arXiv:2206.10839 [cs.IR] https://arxiv.org/abs/2206.10839

[58] Ziqi Yin, Jianyang Gao, Pasquale Balsebre, Gao Cong, and Cheng Long. 2025. DEG: Efficient Hybrid Vector Search Using the Dynamic Edge Navigation Graph. *Proc. ACM Manag. Data* 3, 1, Article 29 (Feb. 2025), 28 pages. https://doi.org/10.1145/3709679

[59] Song Yu, Shengyuan Lin, Shufeng Gong, Yongqing Xie, Ruicheng Liu, Yijie Zhou, Ji Sun, Yanfeng Zhang, Guoliang Li, and Ge Yu. 2025. A Topology-Aware Localized Update Strategy for Graph-Based ANN Index. *Proc. VLDB Endow.* 19, 3 (Nov. 2025), 495–508. https://doi.org/10.14778/3778092.3778108

[60] Konstantinos Zagoris, Avi Arampatzis, and Savvas A. Chatzichristofis. 2010. www.MMRetrieval.net: a multimodal search engine. In *Proceedings of the Third International Conference on SImilarity Search and APplications* (Istanbul, Turkey) *(SISAP '10)*. Association for Computing Machinery, New York, NY, USA, 117–118. https://doi.org/10.1145/1862344.1862363

[61] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, Mao Yang, and Lidong Zhou. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. USENIX Association, Boston, MA, 377–395. https://www.usenix.org/conference/osdi23/presentation/zhang-qianxi

[62] Wayne Xin Zhao, Jing Liu, Ruiyang Ren, and Ji-Rong Wen. 2022. Dense Text Retrieval based on Pretrained Language Models: A Survey. *CoRR* abs/2211.14876 (2022).

[63] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proc. ACM Manag. Data* 2, 1, Article 69 (March 2024), 26 pages. https://doi.org/10.1145/3639324